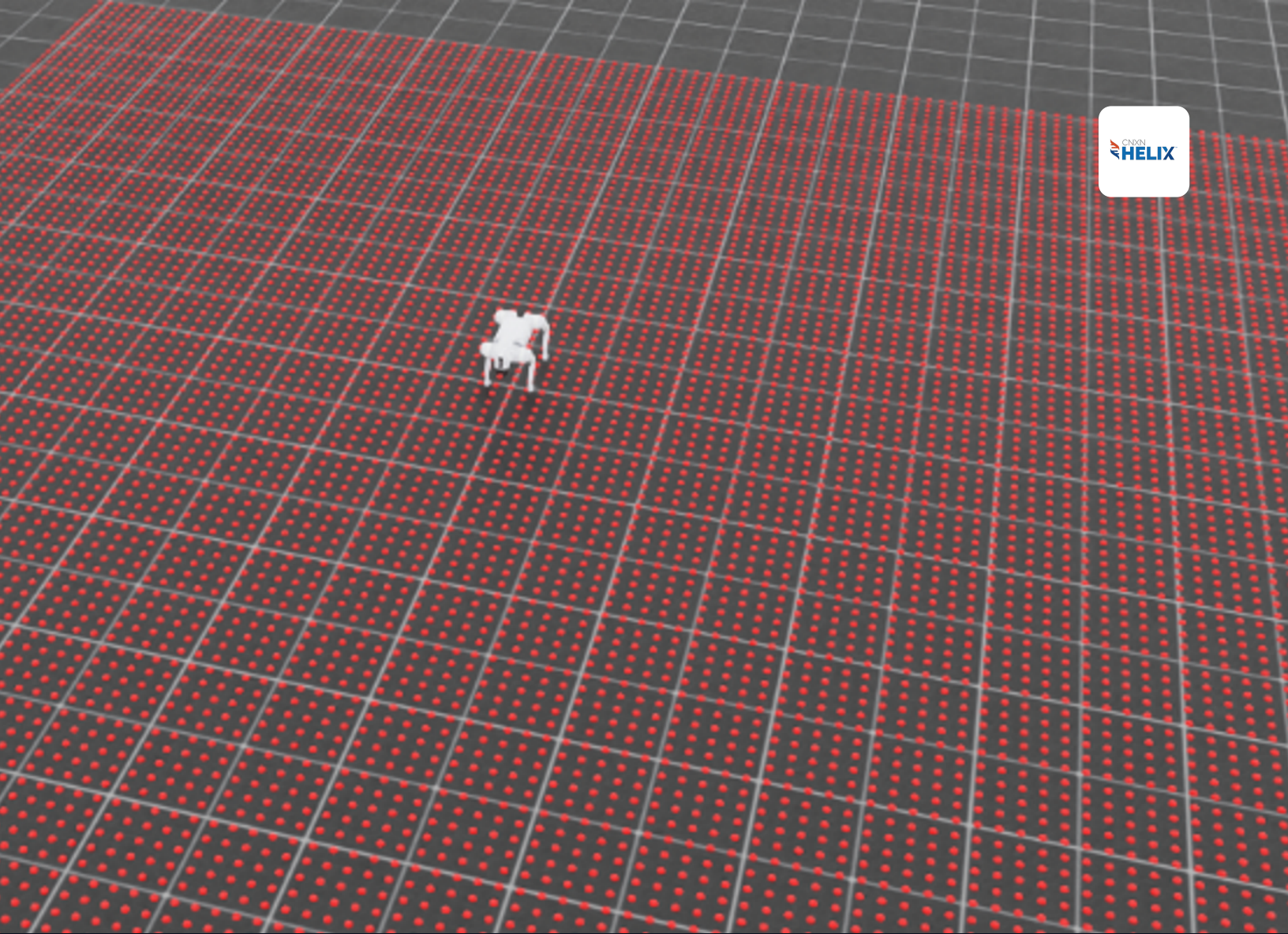




Connection Helix Center for AI and Robotics

Reinforcement-learning locomotion, YOLO-based object detection, and LiDAR navigation for a Unitree Go2 quadruped exploring a simulated warehouse.

Python

Reinforcement Learning

Isaac Lab

ROS2 Humble

YOLOv8

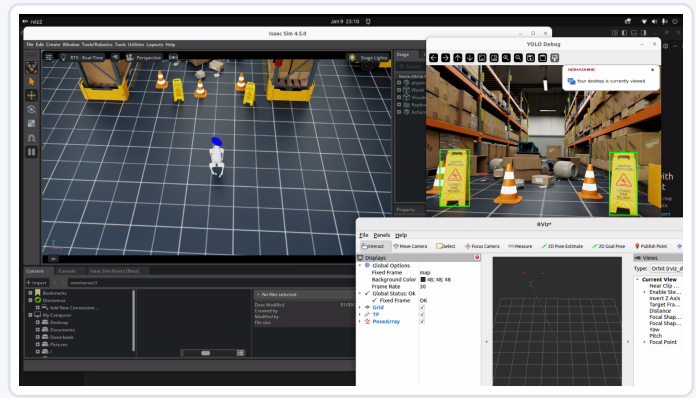
LiDAR

Objective

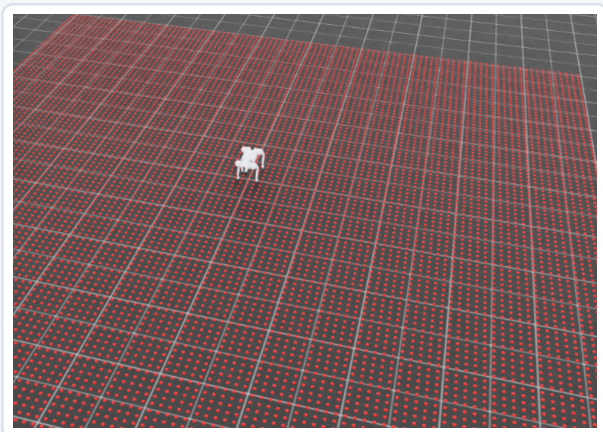
Give a digital twin of the Unitree Go2 quadruped a target object — in this test case, a bright yellow “slipping cone” — and have it navigate a warehouse of sharp turns and narrow corridors to find it autonomously.

YOLOv8 Object Detection & Localization

A YOLOv8 model detects the caution sign from the Go2's RGB-D feed. A custom ROS2 perception node fuses the bounding-box center with depth sampling, temporal filtering, and tf2 coordinate transforms to publish a stable 3D object location in the map frame.



YOLOv8 detection + RViz2 visualization



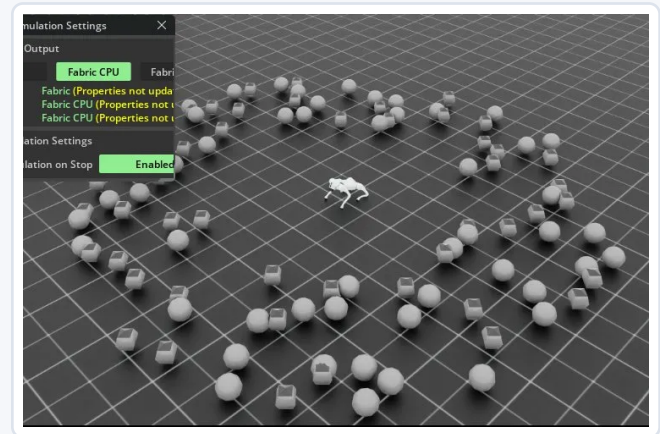
Go2 walking policy during gait training

Reinforcement-Learning Walking Policy

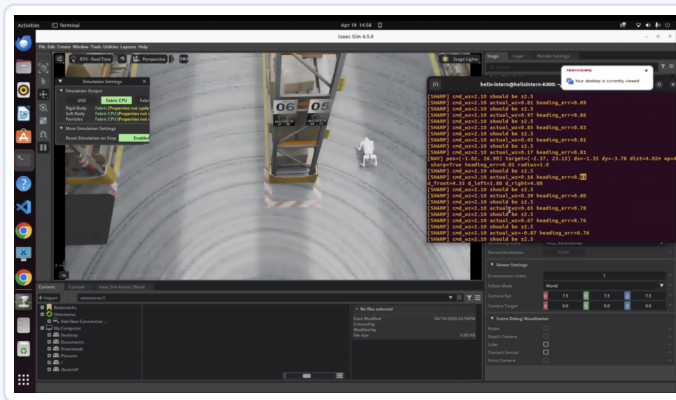
A 12-joint action policy was trained with a reward shaped for forward-velocity tracking, smooth ground contact, and left/right leg symmetry — explicitly penalizing “dead legs” and asymmetric gait to reach a stable, natural walk.

LiDAR-Based Obstacle Avoidance

The point cloud is reduced to front / left / right clearance sectors. When an obstacle enters the safety margin, the policy is rewarded for turning toward whichever side has more open space, then resuming forward motion.



LiDAR obstacle detection during training

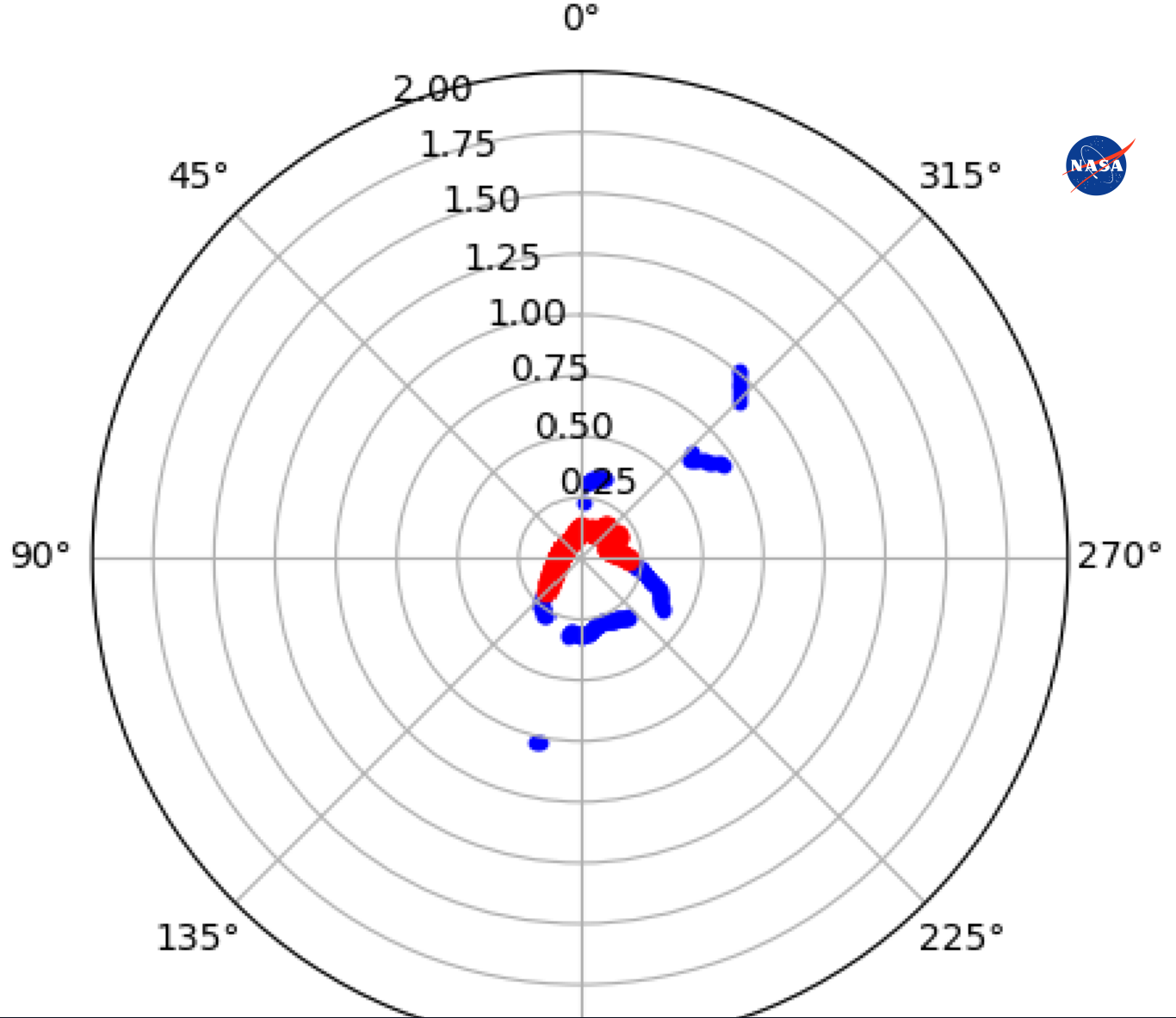


Full navigation policy in the simulated warehouse

The Go2 combines learned locomotion, obstacle avoidance, and waypoint tracking to autonomously traverse a warehouse and locate its target object in simulation.

Waypoint Navigation

Locomotion and avoidance were fused with waypoint guidance: heading error drives yaw commands, corridor-centering keeps the robot off the walls, and sharp-turn logic slows or briefly reverses the robot when boxed in.



NASA Johnson Space Center

A ROS2 LiDAR pipeline built for the MicroChariot rover — reliable streaming, lifecycle-managed drivers, CSV logging, and real-time proximity detection.

Python

LiDAR

ROS2 Foxy

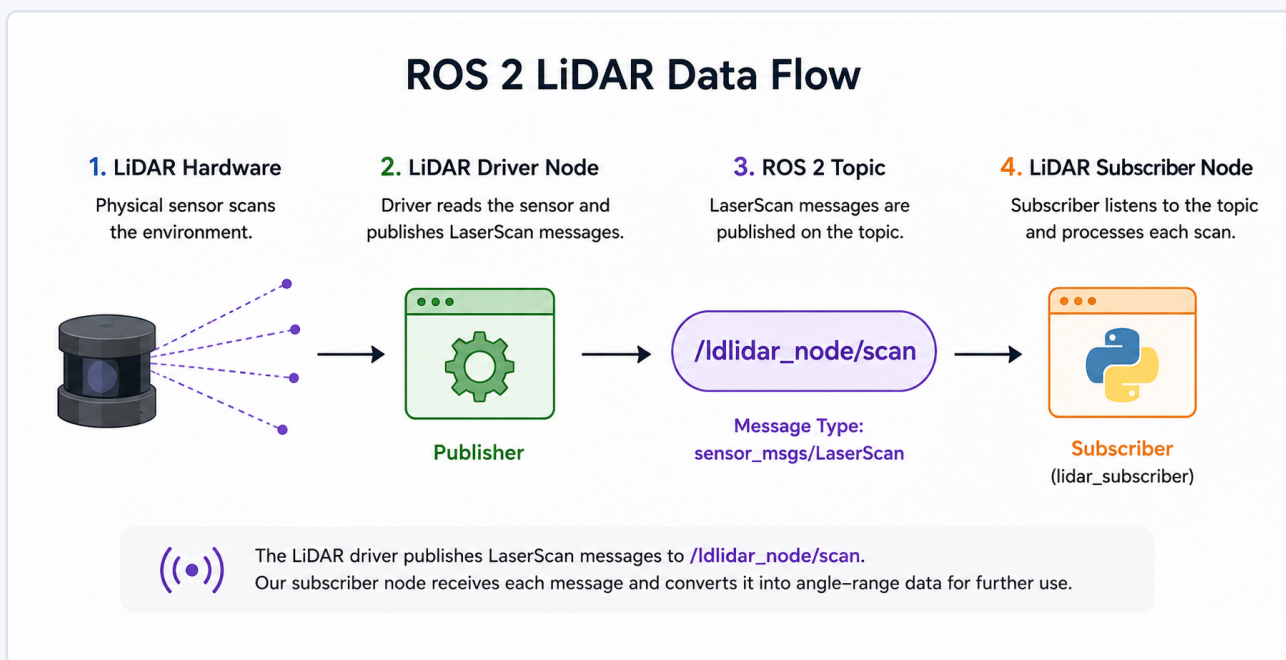
Ubuntu 20.04

Objective

Develop a ROS2 LiDAR pipeline for reading sensor data, logging it to CSV, and detecting nearby objects — intended for integration into MicroChariot's object-avoidance navigation stack.

Publisher / Subscriber Architecture

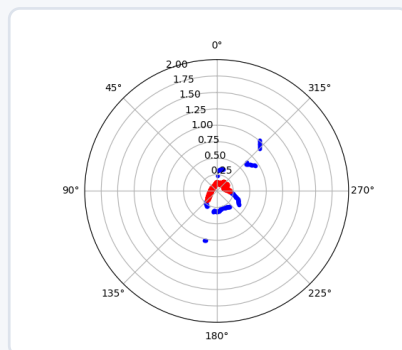
The LiDAR driver publishes each scan as a sensor_msgs/LaserScan message. A dedicated subscriber node decouples the hardware interface from downstream processing, so the same stream can feed multiple ROS2 nodes.



ROS2 data flow: LiDAR hardware -> subscriber node

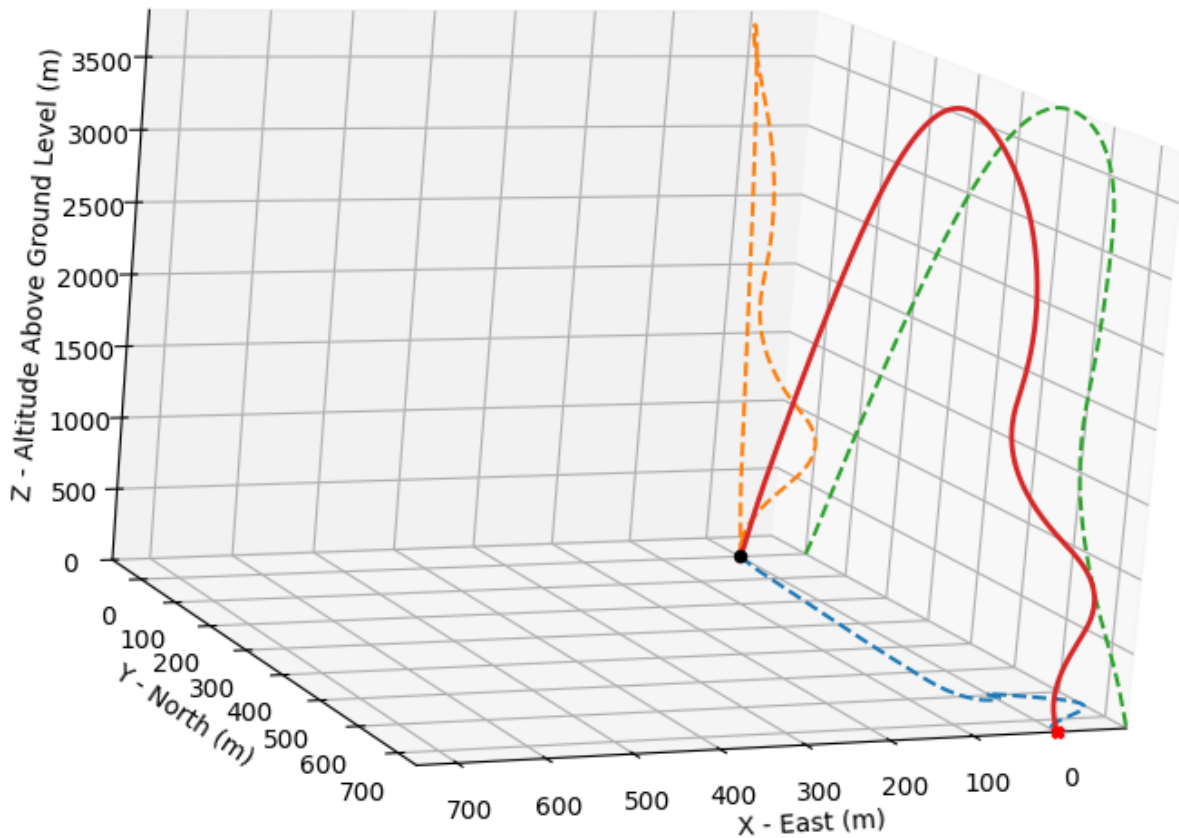
Managed Startup, Shutdown & CSV Logging

The driver runs as a ROS2 lifecycle node, explicitly configured and activated on launch and cleanly deactivated on exit. A rolling 50-scan buffer is rewritten to CSV on every update for downstream analysis.



Polar scan visualization prior to CSV logging

Each LiDAR beam is compared against a 0.20 m proximity threshold; beams inside that radius are flagged.



Longhorn Rocketry Association Simulations

High-fidelity 6-DOF flight simulation and an active airbrake controller built to hit a 10,000 ft apogee target at IREC.

RocketPy

6-DOF EOM

PD Control

Python

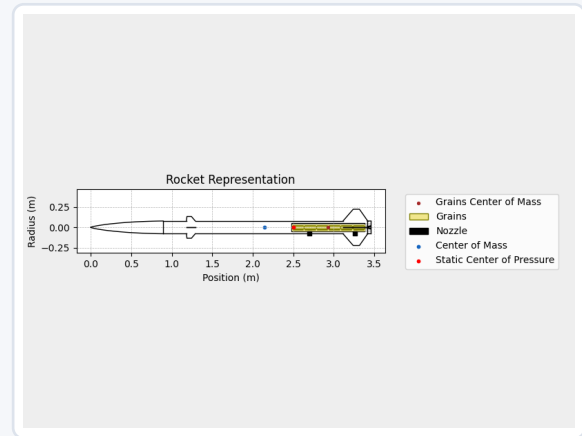
Weather Sounding

Objective

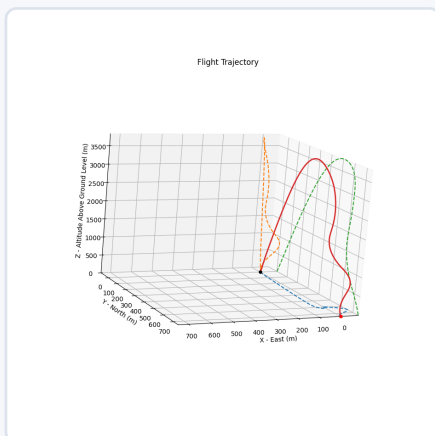
Develop a high-fidelity simulation to accurately predict the apogee and trajectory of LRA's competition rocket, integrating propulsion, mass properties, atmospheric dynamics, and airbrake actuation.

Full-Flight Numerical Simulation

A complete vehicle model — motor, mass properties, nose cone, fin set, canards, and drogue/main parachutes — is assembled and propagated through launch-rail departure, powered ascent, coast, apogee, and recovery.



Rocket geometry: center of mass & center of pressure



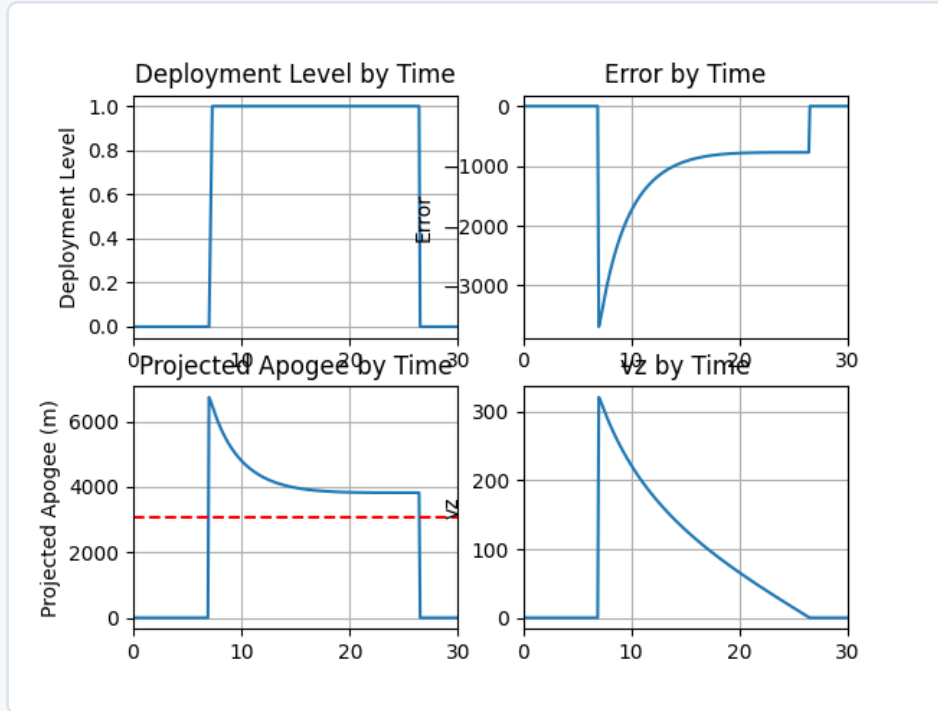
Predicted trajectory and apogee from simulation

Apogee & Trajectory Output

The assembled model produces full trajectory and apogee predictions used to validate the vehicle design against the competition's 10,000 ft target altitude ahead of flight.

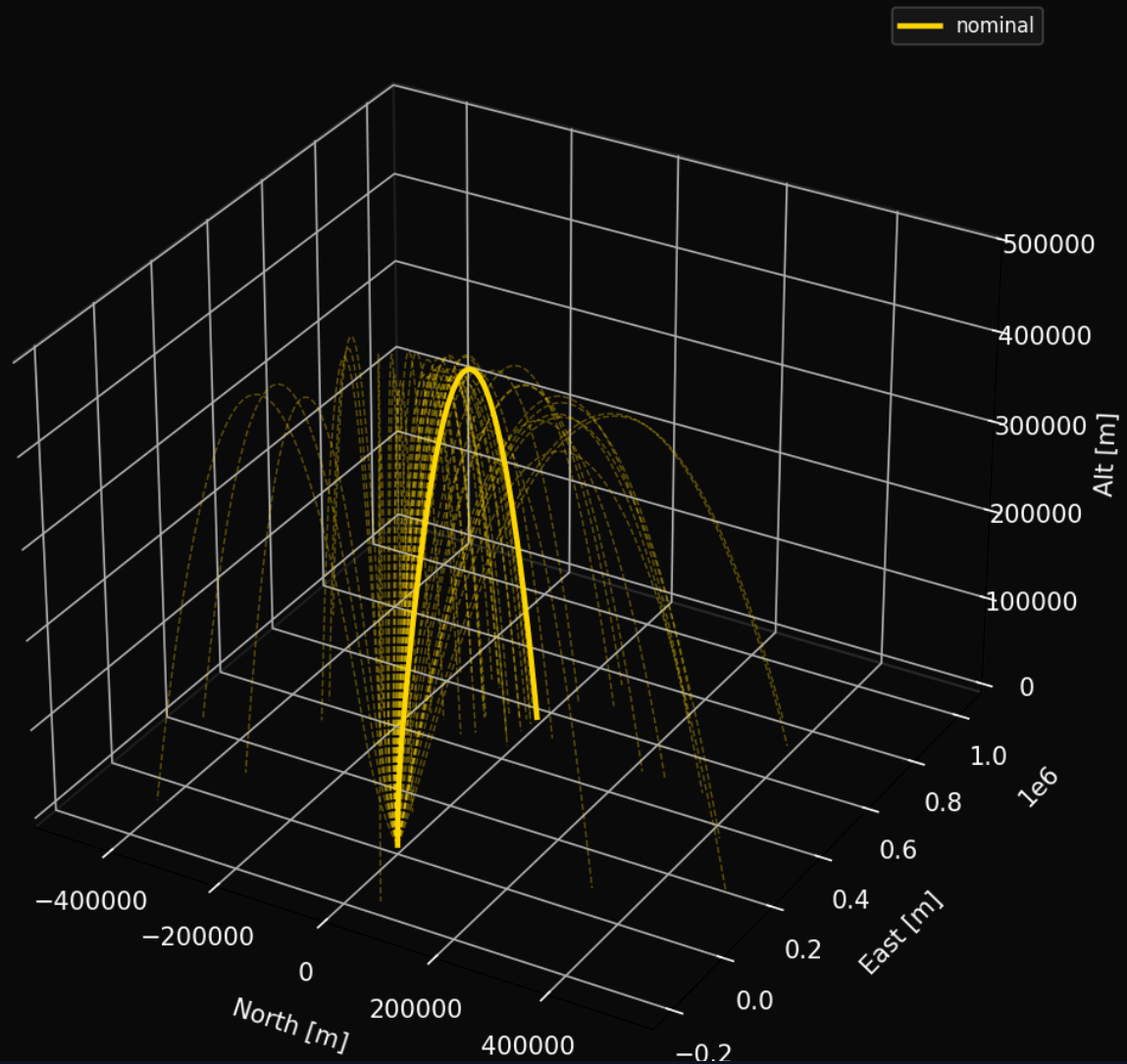
Active Airbrake Deployment

An active airbrake controller continuously estimates projected apogee during ascent from altitude, vertical velocity, air density, mass, and Mach number. When the projection overshoots the 10,000 ft target, a PD controller commands additional airbrake deployment — rate-limited and clipped between fully retracted and fully extended — to bleed off excess energy in real time.



Airbrake deployment response and projected apogee during ascent

Monte Carlo — Falcon 9 Block 5
(50 runs, +/-10% IC + control dispersion)



Project Nova — Physics-Informed Neural Network

A PyTorch surrogate trained on a custom 6-DOF rocket simulator to learn and predict Falcon-9-style trajectory dynamics.

PyTorch

Physics-Informed NN

6-DOF EOM

Monte Carlo

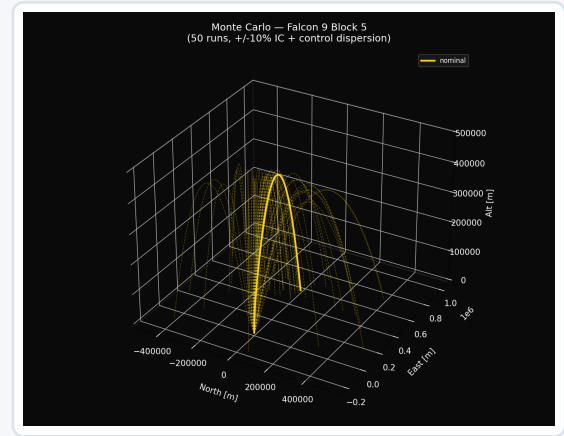
SGD

Objective

Train a neural network to predict the 6-DOF derivatives — rotational velocity, tangential velocity, mass, and more — of a nominal Falcon 9 launch, demonstrating how neural surrogates could accelerate aerospace simulation for more advanced physics models.

6-DOF Simulator & Monte Carlo

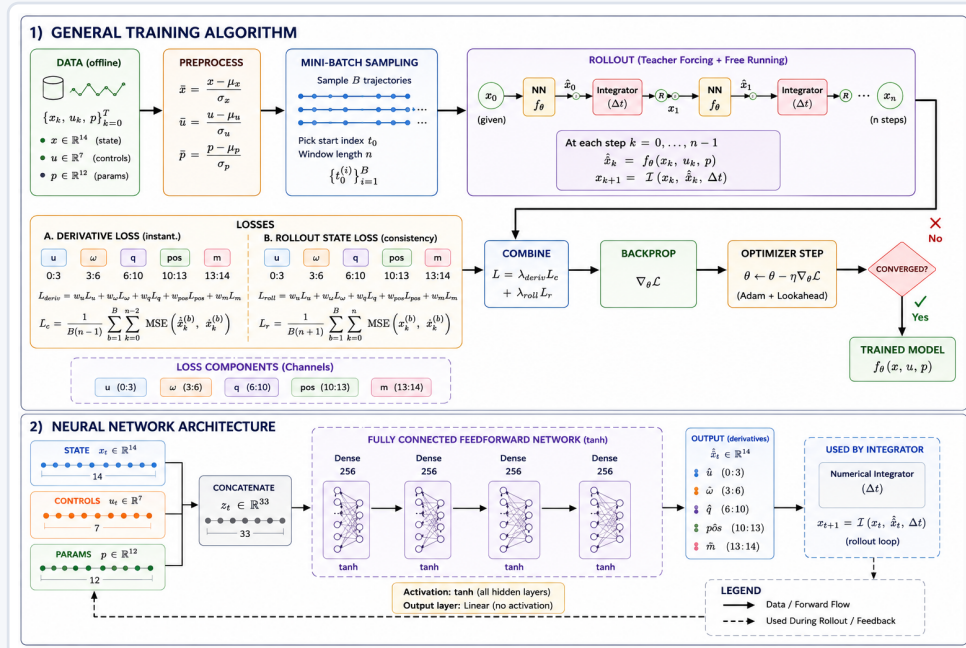
A numerical 6-DOF simulator (14-state, quaternion attitude, RK4 integration) generates training data. Monte Carlo trajectories perturb control inputs around the nominal flight to expose the network to a distribution of flights.



50 Monte Carlo trajectories with perturbed rotations

Physics-Informed Training

A 33-to-14 fully connected network (4 hidden layers, tanh activations) predicts state derivatives, trained on derivative matching, a physics-residual term against the governing equations of motion, and multi-step RK4 rollout loss.



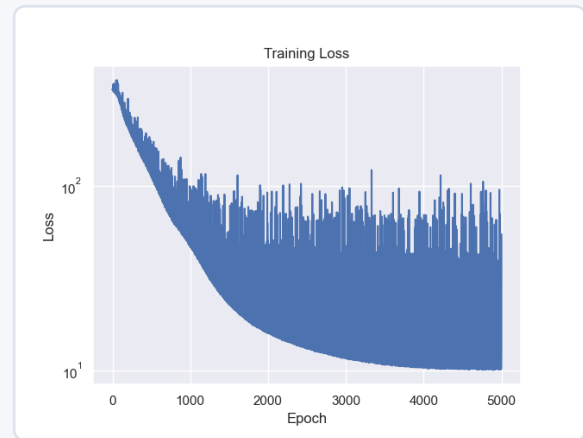
Training pipeline and network architecture

Current Work — Stochastic Optimization

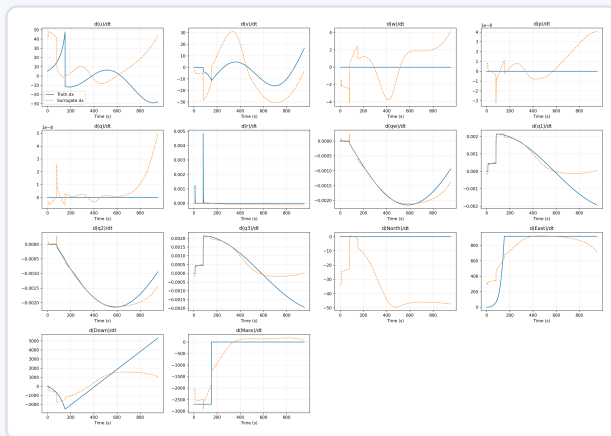
Training uses Grid + Latin Hypercube Sampling to redraw mini-batches every epoch, and a Lookahead optimizer to stabilize Adam's updates. The open challenge is balancing the data-fitting and physics-residual objectives, whose gradients can conflict — tracked via cosine similarity between gradient vectors.

Training Loss Over Epochs

Fluctuations in the loss curve reflect the dynamic, re-sampled gradient produced by the GLHS batching strategy rather than a fixed descent path.



Loss values across training epochs



Prediction accuracy across state derivatives

Quaternion Rotation vs. Other States

Lookahead optimization keeps predictions physically plausible, reaching ~80% accuracy on quaternion rotation while accuracy on other derivatives remains a work in progress.